Visual Inertial SLAM with the Extended Kalman Filter

Dhruv Shah
PhD Student, Mechanical and Aerospace Engineering
University of California, San Diego
dhshah@ucsd.edu

ECE 276A: Sensing & Estimation in Robotics
March 20, 2026

1 Introduction

The goal of this project is to implement visual-inertial simultaneous localization and mapping (SLAM) using an extended Kalman filter (EKF). We utilize data obtained by Clearpath Jackal robots navigating on MIT's campus. The system processes measurements from an inertial measurement unit (IMU) and a stereo camera. The system must fuse the high-frequency motion data from the IMU with the complex, lower-frequency visual data from the stereo cameras to maintain an accurate estimate of the robot's $SE(3)$ trajectory while mapping static 3D landmarks in the environment.

1.1 Importance of the Problem

Accurate state estimation is a prerequisite for downstream robotics tasks such as path planning, obstacle avoidance, and control. While an IMU provides excellent short-term dead-reckoning capabilities by measuring linear and angular velocities, integrating these measurements over time leads to unbounded error accumulation. Conversely, cameras provide rich, absolute structural information about the environment but are computationally expensive to process and operate at lower frequencies. Visual-Inertial SLAM is critical because it leverages the complementary strengths of these two modalities. By continuously tracking visual features and triangulating them into a 3D map, the system can use these static landmarks as spatial anchors to bound the IMU's drift, enabling robust navigation even in GPS-denied environments.

1.2 Overview of the Approach

Our approach tackles the VI-SLAM problem using a tightly coupled Extended Kalman Filter architecture, broken down into three primary stages:

1. **Motion Prediction (Dead-Reckoning):** We implement an EKF prediction step based on the $SE(3)$ kinematics equations and the given linear velocity and angular velocity measurements from the IMU to estimate the pose over time.

2. **Landmark Mapping:** Assuming a locally accurate IMU trajectory, we initialize and map stationary visual features. To manage the immense computational complexity of tracking thousands of visual features, the EKF update step utilizes a banded block-diagonal sparse covariance matrix structure via SciPy's `csr_matrix`. We also employ a strict front-end feature filter to reject features based on epipolar error, minimal disparity, and tracking age.

3. **Pose Correction (VI-SLAM):** Finally, we obtain a complete visual-inertial SLAM algorithm by applying an EKF update step to the IMU pose based on the stereo-camera observation model. Assuming the previously mapped 3D landmark positions are known and static, we utilize the standard EKF update equations to correct the predicted pose mean and covariance matrices, yielding the final pose estimate.

We apply these three steps over time to maintain an accurate estimate of the robot's trajectory and the 3D map of the environment. The system is evaluated on real-world data collected by Clearpath Jackal robots navigating MIT's campus, demonstrating the effectiveness of our visual-inertial SLAM implementation.

2 Problem Formulation

To formulate the Visual-Inertial SLAM problem, we must define the state variables, the system models, and the objective.

Let $T_t \in SE(3)$ denote the pose of the robot (specifically, the transformation from the IMU body frame to the fixed world frame) at time t. Let $m \in \mathbb{R}^{3\times M}$ denote the 3D coordinates of M static landmarks in the world frame. Furthermore, let $z_t \in \mathbb{R}^{4\times M}$ denote the pixel coordinates of the M features observed by the stereo camera at time t.

2.1 Motion Model

The evolution of the robot's pose is driven by the IMU measurements, consisting of linear velocities $v_t \in \mathbb{R}^3$ and angular velocities $\omega_t \in \mathbb{R}^3$, which can be stacked into a twist vector $u_t = [v_t^T, \omega_t^T]^T \in \mathbb{R}^6$. With a time discretization step of τ_t, the nominal kinematics governing the pose transition from time t to $t+1$ are modeled as:

$$T_{t+1} = T_t \exp(\tau_t \hat{u}_t)$$

where $\hat{u}_t \in \mathbb{R}^{4\times 4}$ is the twist matrix in $se(3)$. The uncertainty in this motion model is captured by the perturbation kinematics, which propagate the covariance matrix subject to a Gaussian process noise $w_t \sim \mathcal{N}(0, W)$.

2.2 Observation Model

The stereo camera provides visual observations of the 3D landmarks. In our implementation, we explicitly model the projections into the left and right cameras separately rather than using a unified stereo matrix. Let K_L and K_R denote the intrinsic calibration matrices for the left and right cameras, and let ${}_LT_I \in SE(3)$ and ${}_RT_I \in SE(3)$ denote their respective extrinsic transformations from the IMU frame.

If a landmark $m_j \in \mathbb{R}^3$ is observed at time t, we first transform its homogeneous world coordinates $\underline{m}_j$ into the IMU frame using the inverse pose T_t^{-1}. We then project this point into both camera frames. The predicted stereo pixel observation $\tilde{z}_{t,j} \in \mathbb{R}^4$ is formed by stacking the resulting 2D pixel coordinates:

$$\tilde{z}_{t,j} = \begin{bmatrix} u_L \\ v_L \\ u_R \\ v_R \end{bmatrix} = \begin{bmatrix} K_L \pi \left({}_LT_I T_t^{-1} \underline{m}_j \right) \\ K_R \pi \left({}_RT_I T_t^{-1} \underline{m}_j \right) \end{bmatrix}$$

where $\pi(\cdot)$ represents the perspective projection function that normalizes the 3D coordinates by their depth (Z-axis) to yield the final image plane coordinates.

2.3 Objective

Our ultimate goal is to estimate the pose trajectory T_t and the world-frame coordinates of the robust subset of the M landmarks. We are required to perform this estimation in a causal, recursive manner suitable for online implementation. By utilizing the incoming stream of IMU twists u_t and visual observations z_t through the defined motion and observation models, we aim to maintain a joint belief over the robot pose $\hat{T}_t \in \mathrm{SE}(3)$ and landmark map $\hat{m} \in \mathbb{R}^{3\times M}$ via an Extended Kalman Filter (EKF).

3 Technical Approach

Our visual-inertial SLAM pipeline is constructed from several interacting modules: IMU dead-reckoning, front-end visual processing, EKF landmark mapping, and EKF pose correction.

3.1 Dead-Reckoning (IMU Prediction)

To establish a baseline trajectory without visual corrections, we integrate the IMU measurements using the $SE(3)$ motion model. We represent the prior pose belief as a Gaussian over the Lie group, $T_t|\mathbf{z}_{0:t}, \mathbf{u}_{0:t-1} \sim \mathcal{N}(\boldsymbol{\mu}_{t|t}, \Sigma_{t|t})$, where the nominal mean pose is $\boldsymbol{\mu}_{t|t} \in SE(3)$ and the covariance is $\Sigma_{t|t} \in \mathbb{R}^{6\times 6}$. This implies that the true pose is a local perturbation of the mean, modeled via the exponential map: $T_t = \boldsymbol{\mu}_{t|t} \exp(\hat{\delta\boldsymbol{\mu}}_{t|t})$, where the perturbation twist is normally distributed as $\delta\boldsymbol{\mu}_{t|t} \sim \mathcal{N}(0, \Sigma_{t|t})$.

The IMU provides linear and angular velocities which we stack into a control twist vector $\mathbf{u}_t = [\mathbf{v}_t^\top, \boldsymbol{\omega}_t^\top]^\top \in \mathbb{R}^6$. Given a time discretization step τ_t, the EKF prediction step is governed by two separate kinematic models: the nominal kinematics to propagate the mean, and the perturbation kinematics to propagate the covariance.

The nominal mean pose is updated by integrating the IMU twist via the $SE(3)$ exponential map:

$$\boldsymbol{\mu}_{t+1|t} = \boldsymbol{\mu}_{t|t} \exp(\tau_t \hat{\mathbf{u}}_t)$$

where $\hat{\mathbf{u}}_t \in \mathbb{R}^{4\times 4}$ is the $se(3)$ skew-symmetric matrix representation of the twist.

The local uncertainty is driven by the perturbation kinematics, which includes the process noise $\mathbf{w}_t \sim \mathcal{N}(0, W)$:

$$\delta\boldsymbol{\mu}_{t+1|t} = \exp(-\tau_t \mathbf{u}_t^\curlywedge)\delta\boldsymbol{\mu}_{t|t} + \sqrt{\tau_t}\mathbf{w}_t$$

Taking the expectation $\mathbb{E}[\delta\boldsymbol{\mu}_{t+1|t}\delta\boldsymbol{\mu}_{t+1|t}^\top]$ yields the covariance prediction equation:

$$\Sigma_{t+1|t} = \exp(-\tau_t \mathbf{u}_t^\curlywedge)\Sigma_{t|t} \exp(-\tau_t \mathbf{u}_t^\curlywedge)^\top + \tau_t W$$

Operator Definitions: The mappings from the 6D twist $\mathbf{u}_t$ to the 4×4 and 6×6 matrix spaces are explicitly defined as follows:

$$\mathbf{u}_t = \begin{bmatrix} \mathbf{v}_t \\ \boldsymbol{\omega}_t \end{bmatrix} \in \mathbb{R}^6, \quad \hat{\mathbf{u}}_t = \begin{bmatrix} \hat{\boldsymbol{\omega}}_t & \mathbf{v}_t \\ \mathbf{0}^\top & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}, \quad \mathbf{u}_t^\curlywedge = \begin{bmatrix} \hat{\boldsymbol{\omega}}_t & \hat{\mathbf{v}}_t \\ \mathbf{0} & \hat{\boldsymbol{\omega}}_t \end{bmatrix} \in \mathbb{R}^{6\times 6}$$

Here, $(\cdot)^\wedge$ maps a 3D vector to its 3×3 skew-symmetric form, and $(\cdot)^\curlywedge$ maps the 6D twist to its 6×6 adjoint matrix representation.

3.2 Front-End Visual Processing

Raw stereo tracking data often contains noise, feature mismatches, and dynamic objects. Before incorporating the visual measurements $z_t \in \mathbb{R}^{4\times M}$ into the EKF, we must filter out unreliable data to maintain map integrity and prevent the filter from diverging.

3.2.1 Feature Filtering

At any given timestep t, a stereo observation of a landmark is represented by its pixel coordinates $z_t = [u_L, v_L, u_R, v_R]^\top$. To be considered an "elite" feature suitable for state estimation, the track must satisfy three strict geometric and temporal criteria:

1. **Epipolar Constraint (Vertical Difference):** For a perfectly rectified stereo camera pair, corresponding features must lie on the same horizontal epipolar line, meaning their vertical pixel coordinates should be identical. We enforce this constraint by checking the vertical disparity:
$$|v_L - v_R| \leq \epsilon_y$$
where ϵ_y is a small pixel threshold (e.g., 1.0 pixel). If the difference exceeds this threshold, the feature is likely a false match from the optical flow algorithm or a dynamically moving object, and is discarded.

2. **Depth Observability (Minimum Disparity):** The depth Z of a triangulated feature is inversely proportional to its horizontal disparity $d = u_L - u_R$. If a feature is located extremely far from the camera, its disparity approaches zero, causing the depth estimate to approach infinity ($Z \to \infty$). This leads to severe numerical instability in the EKF Jacobians. To prevent this, we enforce a minimum disparity threshold:
$$u_L - u_R \geq d_{min}$$
Features failing this check are deemed too distant to provide reliable parallax and are dropped.

3. **Temporal Consistency (Feature Age):** To ensure we are mapping static, reliable landmarks rather than transient reflections or noise, a feature must survive tracking for a minimum continuous lifespan. A feature i is only accepted if it has valid observations for every frame in a sliding window of size t_{age}:
$$z_{\tau,i} \neq [-1,-1,-1,-1]^\top \quad \forall \tau \in [t - t_{age}, t]$$
This age-based filtering guarantees that the EKF only updates its state using robust, well-established environmental anchors.

3.2.2 Landmark Initialization

For features that survive the filtering stage but are not yet part of the map, we must estimate their initial 3D positions in the fixed world frame. This is achieved through stereo triangulation and sequential frame transformations.

First, we determine the stereo baseline b (the physical distance between the two optical centers). Given the extrinsic transformations from the IMU frame to the left and right cameras, ${}_LT_I, {}_RT_I \in SE(3)$, the relative transformation from the right camera to the left camera is computed as:
$${}_LT_R = {}_LT_I \cdot {}_RT_I^{-1}$$

The baseline b is simply the Euclidean norm of the translational component of ${}_LT_R$.

Next, we triangulate the depth Z of the feature relative to the left camera. Using the left camera's focal length f (extracted from the intrinsic matrix K_L) and the horizontal disparity $d = u_L - u_R$, the depth is estimated as:

$$Z = \frac{f \cdot b}{d}$$

With the depth known, we back-project the 2D pixel coordinates (u_L, v_L) into a 3D point in the left camera's optical frame. Using the optical center (c_u, c_v) from K_L, the spatial coordinates are:

$$X = \frac{(u_L - c_u)Z}{f}, \quad Y = \frac{(v_L - c_v)Z}{f}$$

This yields the homogeneous coordinate vector in the left camera frame, $\underline{m}_L = [X, Y, Z, 1]^\top$.

Finally, we must transform this locally-observed point into the global world frame so it can be tracked as a static geometric anchor. We map the point from the left camera frame to the IMU body frame using the inverse of the left extrinsic matrix (${}_LT_I^{-1}$), and then map it from the IMU frame to the world frame using the current IMU pose estimate $T_t \in SE(3)$. The fully initialized landmark in the world frame, $\underline{m}_W$, is given by:

$$\underline{m}_W = T_t \cdot {}_LT_I^{-1} \cdot \underline{m}_L$$

The Cartesian x, y, and z components of $\underline{m}_W$ are extracted and added to the EKF map state $\mu_{landmarks}$, allowing the filter to update its position and utilize it for future pose corrections.

3.3 EKF Landmark Mapping (Update)

Once the IMU pose prediction $\mu_{t+1|t} \in SE(3)$ is established, we assume it to be locally accurate and use it to refine the 3D map. Our prior belief over the M mapped landmarks is represented by a Gaussian distribution with mean $\boldsymbol{\mu}_t \in \mathbb{R}^{3M}$ and covariance $\Sigma_t \in \mathbb{R}^{3M \times 3M}$.

For every valid visual feature observed at time $t+1$, we compute its predicted stereo observation $\tilde{\mathbf{z}}_{t+1,i}$ based on the prior mean $\boldsymbol{\mu}_t$. In our implementation, we calculate the projections into the left and right cameras independently using their distinct calibration matrices, K_L and K_R, and extrinsic transformations, ${}_LT_I$ and ${}_RT_I$:

$$\tilde{\mathbf{z}}_{t+1,i} = \begin{bmatrix} K_L \pi \left({}_LT_I \mu_{t+1|t}^{-1} \underline{\boldsymbol{\mu}}_{t,j} \right) \\ K_R \pi \left({}_RT_I \mu_{t+1|t}^{-1} \underline{\boldsymbol{\mu}}_{t,j} \right) \end{bmatrix} \in \mathbb{R}^4$$

where $\underline{\boldsymbol{\mu}}_{t,j} \in \mathbb{R}^4$ is the homogeneous representation of the j-th landmark, and $j = \Delta_{t+1}(i)$ denotes the known data association correspondence.

The Jacobian $H_{t+1,i,j}$ of the predicted observation with respect to the j-th landmark $\mathbf{m}_j$, evaluated at $\boldsymbol{\mu}_{t,j}$, is constructed by stacking the individual camera Jacobians. When a valid correspondence exists ($\Delta_t(j) = i$), the block Jacobian is defined as:

$$H_{t+1,i,j} = \begin{bmatrix} K_L \frac{d\pi}{d\mathbf{q}} \left({}_LT_I \mu_{t+1|t}^{-1} \underline{\boldsymbol{\mu}}_{t,j} \right) {}_LT_I \mu_{t+1|t}^{-1} P^\top \\ K_R \frac{d\pi}{d\mathbf{q}} \left({}_RT_I \mu_{t+1|t}^{-1} \underline{\boldsymbol{\mu}}_{t,j} \right) {}_RT_I \mu_{t+1|t}^{-1} P^\top \end{bmatrix} \in \mathbb{R}^{4 \times 3}$$

where $P^\top \in \mathbb{R}^{4 \times 3}$ is a projection matrix that drops the homogeneous coordinate. If $\Delta_t(j) \neq i$, the block $H_{t+1,i,j}$ is $\mathbf{0}$. These blocks are concatenated into the full, highly sparse observation matrix $H_{t+1} \in \mathbb{R}^{4N_{t+1} \times 3M}$.

3.3.1 Sparse Covariance EKF Update

Tracking thousands of landmarks introduces a significant computational bottleneck. To maintain scalability, we enforce a banded-diagonal structure on the joint landmark covariance matrix $\Sigma_t \in \mathbb{R}^{3M \times 3M}$, assuming each landmark is only cross-correlated with its k nearest neighbors (e.g., $k = 50$).

We utilize SciPy's `csr_matrix` to handle the sparse matrix operations efficiently. The EKF update step follows the standard linear Kalman filter equations:

$$K_{t+1} = \Sigma_t H_{t+1}^\top \left(H_{t+1} \Sigma_t H_{t+1}^\top + I \otimes V \right)^{-1}$$

$$\boldsymbol{\mu}_{t+1} = \boldsymbol{\mu}_t + K_{t+1}(\mathbf{z}_{t+1} - \tilde{\mathbf{z}}_{t+1})$$

$$\Sigma_{t+1} = (I - K_{t+1} H_{t+1}) \Sigma_t$$

where $I \otimes V$ is the block-diagonal observation noise matrix. Crucially, when updating the sparse covariance matrix Σ_{t+1}, we extract only the structural indices of the original sparse matrix from the decrement term $(K_{t+1} H_{t+1} \Sigma_t)$, entirely avoiding the instantiation of dense $3M \times 3M$ matrices during the update.

3.4 EKF Pose Correction (Update)

Once the map is updated, we assume the mapped landmark locations $\mathbf{m} \in \mathbb{R}^{3M}$ are fixed and known, and we use them to correct the IMU pose drift. Our prior belief for this step is the IMU dead-reckoning prediction, represented by a Gaussian over $SE(3)$ with mean $\boldsymbol{\mu}_{t+1|t}$ and a 6×6 covariance $\Sigma_{t+1|t}$.

To perform the EKF update, we must compute the Jacobian of the predicted observation $\tilde{\mathbf{z}}_{t+1,i}$ with respect to the pose T_{t+1}, evaluated at the prior mean $\boldsymbol{\mu}_{t+1|t}$. Because the state is a transformation matrix in $SE(3)$, taking the derivative with respect to a local twist perturbation requires the circle-dot operator, $(\cdot)^\odot$, which maps a homogeneous point to a 4×6 matrix generator.

For the j-th landmark represented in homogeneous coordinates as $\underline{\mathbf{m}}_j$, the stacked Jacobian for our separated left and right camera model is constructed as:

$$H_{t+1,i} = - \begin{bmatrix} K_L \frac{d\pi}{d\mathbf{q}} \left({}_L T_I \boldsymbol{\mu}_{t+1|t}^{-1} \underline{\mathbf{m}}_j \right) {}_L T_I \left(\boldsymbol{\mu}_{t+1|t}^{-1} \underline{\mathbf{m}}_j \right)^\odot \\ K_R \frac{d\pi}{d\mathbf{q}} \left({}_R T_I \boldsymbol{\mu}_{t+1|t}^{-1} \underline{\mathbf{m}}_j \right) {}_R T_I \left(\boldsymbol{\mu}_{t+1|t}^{-1} \underline{\mathbf{m}}_j \right)^\odot \end{bmatrix} \in \mathbb{R}^{4 \times 6}$$

These individual block Jacobians are evaluated for all N_{t+1} actively observed landmarks and stacked vertically into the full observation matrix $H_{t+1} \in \mathbb{R}^{4N_{t+1} \times 6}$.

With the full Jacobian constructed, we compute the Kalman Gain $K_{t+1} \in \mathbb{R}^{6 \times 4N_{t+1}}$ using the block-diagonal observation noise covariance $I \otimes V$:

$$K_{t+1} = \Sigma_{t+1|t} H_{t+1}^\top \left(H_{t+1} \Sigma_{t+1|t} H_{t+1}^\top + I \otimes V \right)^{-1}$$

The 6×6 IMU pose covariance is then updated using the standard linear formulation:

$$\Sigma_{t+1|t+1} = (I - K_{t+1} H_{t+1}) \Sigma_{t+1|t}$$

Finally, we compute the 6-dimensional twist innovation by multiplying the Kalman Gain with the measurement residual. This twist is mapped back onto the $SE(3)$ manifold via the exponential map to correct the predicted pose mean:

$$\boldsymbol{\mu}_{t+1|t+1} = \boldsymbol{\mu}_{t+1|t} \exp\left((K_{t+1}(\mathbf{z}_{t+1} - \tilde{\mathbf{z}}_{t+1}))^\wedge \right)$$

3.5 VI-SLAM Integration and Tuning

Our complete Visual-Inertial SLAM algorithm brings together the individual prediction and update models into a cohesive, sequential pipeline. At a high level, the system processes each new timestep by advancing through a specific order of operations designed to maintain map stability and trajectory accuracy.

First, incoming visual features undergo strict filtering based on their tracking age, epipolar error, and stereo disparity. This front-end step ensures that only high-quality, static points are passed to the filter. Next, the system uses the IMU's velocity measurements to predict the robot's new position and orientation, acting as our dead-reckoning baseline.

To ensure the system remains stable, we implement a "burn-in" period during the initial sequence of frames. During this phase, the robot's trajectory is determined entirely by the IMU's dead-reckoning. This deliberate delay allows the system to build a trustworthy, well-initialized map of 3D landmarks before we allow those landmarks to influence the robot's pose.

After the burn-in period concludes, the system begins its full closed-loop operation. It first uses the visual observations of already-mapped landmarks to correct the IMU's predicted pose. With the pose successfully corrected, the algorithm then uses that updated pose to refine the 3D positions of the existing landmarks in the map. Finally, any newly discovered features that passed the initial filtering criteria are triangulated and added to the map to be tracked in subsequent frames.

3.6 Filter Tuning and Innovation Analysis

The performance and accuracy of an Extended Kalman Filter (EKF) depend heavily on the proper tuning of the process noise covariance W and the observation noise covariance V. To rigorously tune these parameters for a state operating on the $SE(3)$ Lie group, we analyze the autocorrelation of the state-space innovation.

3.6.1 The Mathematics of the Twist Innovation

In standard linear Kalman filtering, the innovation (or residual) is simply the difference between the actual measurement and the predicted measurement, $r_{t+1} = z_{t+1} - \tilde{z}_{t+1}$. However, because our pose state μ is a transformation matrix in $SE(3)$, we cannot simply add a linear correction to it. Instead, the update is applied via the exponential map:

$$\mu_{t+1|t+1} = \mu_{t+1|t} \exp\left((K_{t+1}(z_{t+1} - \tilde{z}_{t+1}))^\wedge\right)$$

The term inside the exponential map represents our state-space innovation, defined as a 6D twist vector $\Delta\mu_{t+1} \in \mathbb{R}^6$:

$$\Delta\mu_{t+1} = K_{t+1}(z_{t+1} - \tilde{z}_{t+1})$$

Here, the Kalman Gain K_{t+1} acts as a mapping matrix that projects the high-dimensional pixel residual down into a 6-dimensional twist in the tangent space $se(3)$. The $(\cdot)^\wedge$ operator then converts this twist into a 4×4 skew-symmetric matrix so that the exponential map can apply it as a geometric transformation to the prior pose.

3.6.2 The White Noise Property

In optimal estimation theory, a perfectly tuned Kalman filter extracts all available structural information from the measurements. If the filter is optimal, the measurement residual must be a zero-mean white noise sequence, representing the pure "new information" at that specific timestep.

Consequently, our twist vector $\Delta\mu_{t+1}$, which is a linear transformation of that residual, must also behave as a white noise sequence.

We use the autocorrelation of this 6D twist to diagnose the filter's performance:

- **The Ideal Case (Delta Function):** If the filter is perfectly tuned, the current twist correction $\Delta\mu_{t+1}$ will have no correlation with the previous twist correction $\Delta\mu_t$. The autocorrelation plot for each of the 6 components will exhibit a Dirac delta function profile—a spike of 1 at lag 0, and near zero everywhere else.
- **The Suboptimal Case (Colored Noise):** If the autocorrelation shows distinct peaks at lags greater than zero, the filter is exhibiting temporal correlation. This indicates a biased filter. For example, a positive autocorrelation in the forward-translation component implies that the visual update is repeatedly dragging the robot forward frame after frame, suggesting the IMU motion model systematically under-predicts the forward velocity.

3.6.3 Balancing W and V

By plotting the autocorrelation of these 6 twist components, we directly visualize the balance of trust between the IMU (governed by W) and the stereo cameras (governed by V).

- If W is too small (over-trusting the IMU), the filter resists camera measurements, causing visual corrections to smear out over time and yielding high autocorrelation.
- If V is too small (over-trusting the cameras), the filter over-corrects to every pixel noise artifact, leading to chaotic, high-variance adjustments.

By iteratively adjusting W and V to force the autocorrelation of $\Delta\mu_{t+1}$ to resemble a Dirac delta function, we ensure the system is optimally balanced, resulting in a minimum-variance trajectory.

4 Results

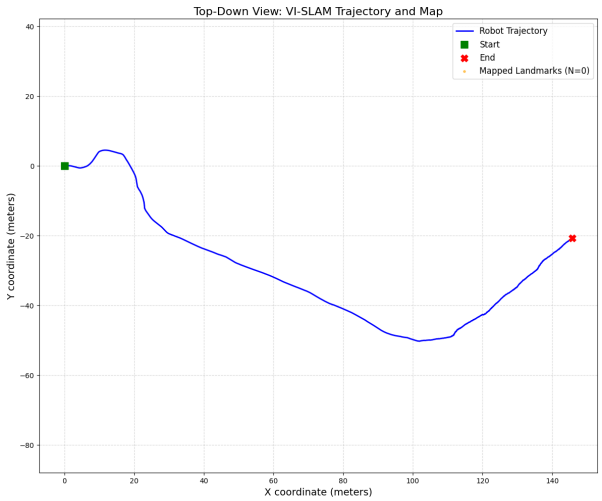
This section presents the results of running the Visual-Inertial SLAM pipeline on both `dataset00` and `dataset01`. We evaluate the performance across both environments by contrasting the dead-reckoning trajectories against the full SLAM solutions, and by analyzing the internal consistency of the filter through covariance and innovation autocorrelation plots.

4.1 Trajectory and Mapping Evaluation

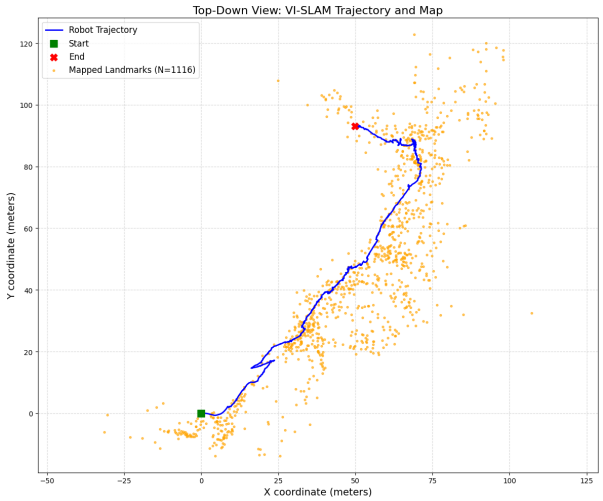
We first demonstrate the necessity of the SLAM algorithm by analyzing the robot's trajectory estimates. Figures 1 and 2 compare the open-loop dead-reckoning against the full EKF SLAM results for `dataset00` and `dataset01`, respectively.

As shown in Figures 1a and 2a, relying solely on the integration of IMU measurements causes immediate and catastrophic drift in both datasets. The estimated paths become nonsensical, drifting rapidly and looping back over themselves chaotically as the IMU noise integrates unbounded over time. This highlights that dead-reckoning alone is entirely insufficient for navigation on these platforms.

Conversely, Figures 1b and 2b show the full tightly coupled VI-SLAM outputs. By fusing stereo visual features with the IMU, the EKF successfully bounds the IMU drift in both scenarios. The resulting trajectories are smooth, coherent, and align with the geometric structures suggested by the dense maps of initialized landmark positions (shown in red). The successful closure of the loop paths across different runs demonstrates the algorithm's robustness to varying environmental conditions.

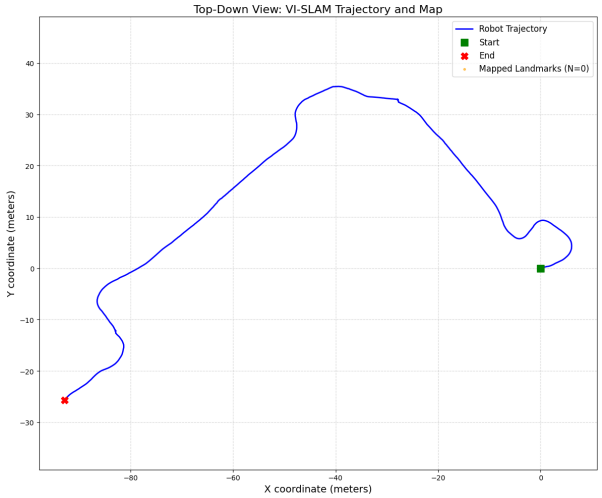


(a) Dead-Reckoning Trajectory

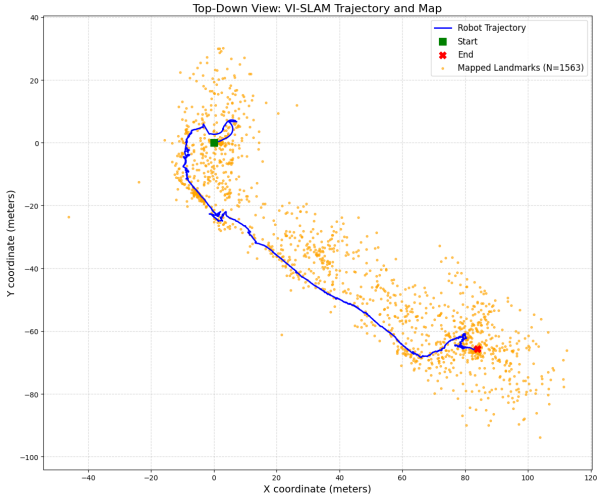


(b) EKF SLAM Final Output

Figure 1: Trajectory evaluation for `dataset00`. The comparison between (a) and (b) clearly demonstrates that visual landmark updates successfully constrain the severe drift inherent in pure inertial dead-reckoning, resulting in a stable and meaningful map and trajectory.



(a) Dead-Reckoning Trajectory



(b) EKF SLAM Final Output

Figure 2: Trajectory evaluation for `dataset01`. In (a), the trajectory has a massive heading error which leads to predictions in the negative x direction, while the true trajectory is in the positive x direction. In (b), the EKF SLAM output successfully corrects this heading error and produces a stable trajectory that aligns with the map of landmarks.

4.2 Filter Stability and Tuning Consistency

To prove that the EKF is optimally tuned and stable across multiple datasets, we look at internal consistency metrics.

Figures 3 and 4 display the uncertainties (diagonal terms of the covariance matrix) for the three translational and three rotational degrees of freedom over time. Unlike dead-reckoning, where uncertainty grows explosively, here the visual updates constantly correct the state, keeping the

variances bounded and stable throughout the entire run for both datasets. This behavior confirms that the filter remains convergent and reliable regardless of the specific trajectory.

Finally, we analyze the innovation twist autocorrelation in Figures 5 and 6. As discussed in the technical approach, a perfectly tuned filter should yield white noise innovations, meaning their autocorrelation should resemble a Dirac delta function. Because the robot navigates a predominantly planar environment, our analysis primarily focuses on the most observable degrees of freedom: the forward and lateral translations (v_x and v_y) and the yaw rotation (ω_z).

While the autocorrelations largely exhibit the desired sharp spike at lag 0, there are some deviations from pure white noise. For `dataset00`, a minor correlation is visible in the forward velocity component v_x, which is likely attributable to linearization errors inherent in the Extended Kalman Filter. This effect is more pronounced in `dataset01`, where we observe minor correlations in the forward velocity (v_x) and the yaw rotation (ω_z). This increase in correlation is expected, as `dataset01` features a more aggressive trajectory than `dataset00`, which exacerbates the EKF's linearization approximations. Despite these minor artifacts, the overall profiles provide strong validation that we have effectively balanced the IMU process noise (W) and observation noise (V) to extract consistent, maximum information from the sensors.

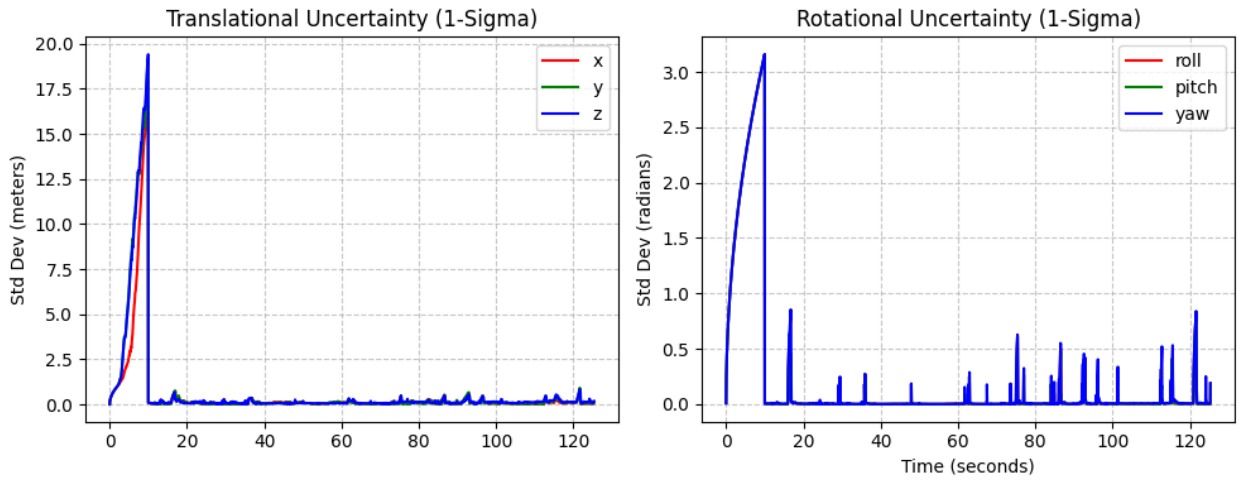


Figure 3: Pose uncertainty (standard deviation) over time for `dataset00`. The bounded and stable variances across all 6 degrees of freedom demonstrate that the filter is stable and convergent. Occasionally, there are spikes in the rotational components, but the translational variances remain well-controlled, which is crucial for maintaining a stable trajectory.

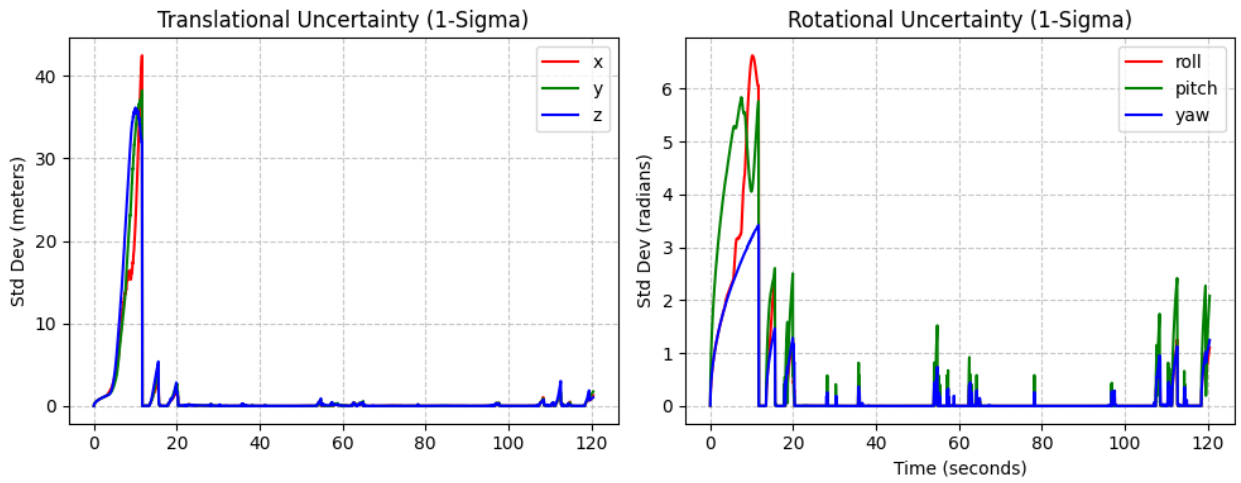


Figure 4: Pose uncertainty (standard deviation) over time for `dataset01`. The bounded and stable variances across all 6 degrees of freedom demonstrate that the filter is stable and convergent. Occasionally, there are spikes in the rotational components, but the translational variances remain well-controlled, which is crucial for maintaining a stable trajectory.

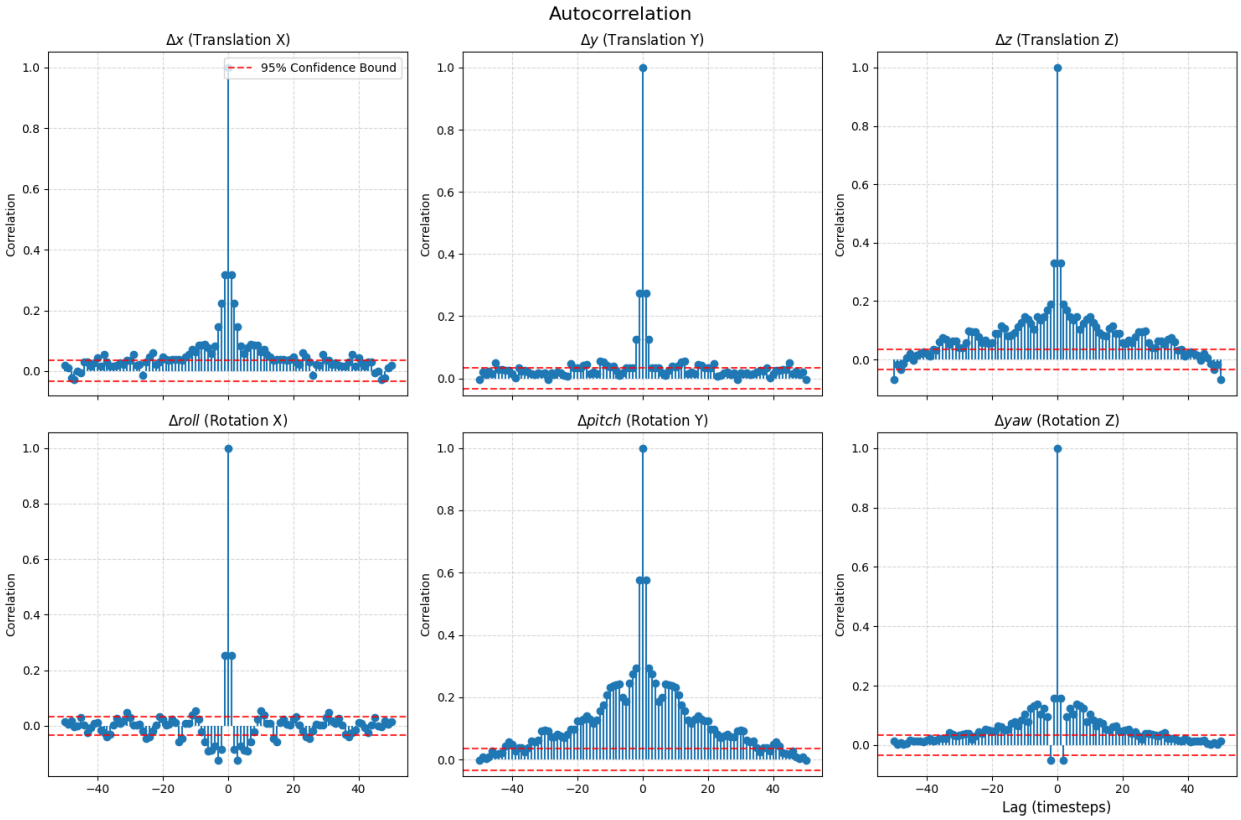


Figure 5: Autocorrelation of the 6D innovation twist components for `dataset00`. We care about the x and y translation components in particular, and the yaw rotation component, since those are the most observable degrees of freedom in a planar environment. There is some minor correlation in the forward velocity component v_x likely due linearization errors in the EKF.

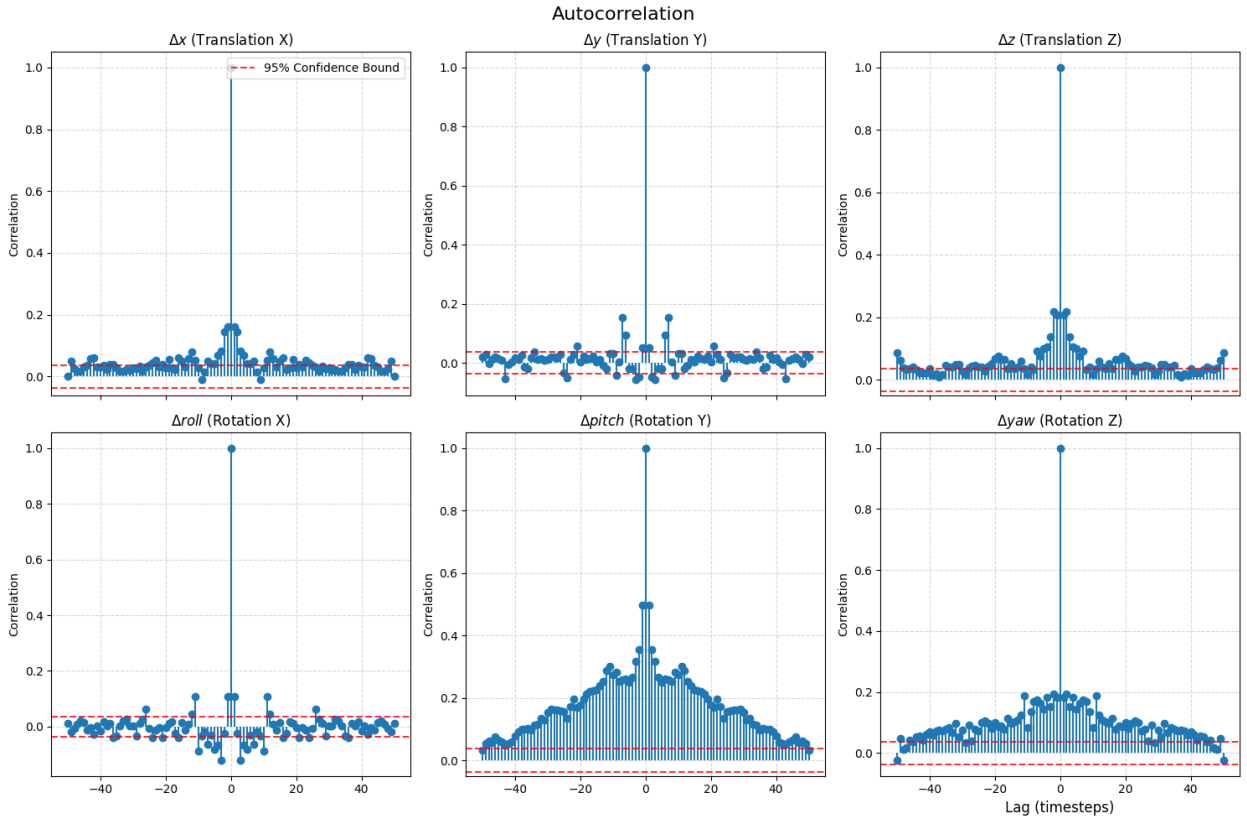


Figure 6: Autocorrelation of the 6D innovation twist components for `dataset01`. We care about the x and y translation components in particular, and the yaw rotation component, since those are the most observable degrees of freedom in a planar environment. There is some minor correlation in the forward velocity component v_x and yaw rotation component ω_z likely due linearization errors in the EKF. Furthermore, the trajectory in `dataset01` is more aggressive than `dataset00`, which may also contribute to the increased correlation in these components.